MAD 1 FLASK RESTFUL

1. Introduction to REST APIs

- **What is an API?**
  - API (Application Programming Interface) allows communication between different software applications.
- **What is a REST API?**
  - REST (Representational State Transfer) is an architectural style for designing networked applications.
  - Uses **HTTP methods**:
    - `GET`: Retrieve data
    - `POST`: Send data
    - `PUT`: Update data
    - `DELETE`: Remove data
- **Why use Flask-RESTful?**
  - Simplifies API creation in **Flask** with built-in support for request parsing, error handling, and serialization.

Basics:

1. `Resource` (from flask_restful)

- `Resource` is a class-based approach to creating API endpoints in Flask-RESTful.
- Each resource represents a specific API endpoint (like `/user`, `/product`).
- You define methods like `get()`, `post()`, `put()`, and `delete()` inside a `Resource` class to handle different HTTP request types.

Example Usage:

- A `User` resource can have a `get()` method to retrieve user data and a `post()` method to create a new user.

---

2. `Api` (from flask_restful)

- `Api` is the core component that turns a Flask app into a RESTful API.
- It manages all the `Resource` classes and routes them to specific URLs.

Key Features:

- Registers resources with their corresponding endpoints.
- Provides automatic error handling for API exceptions.
- Converts Python dictionaries into JSON responses.

Example Usage:

- `api.add_resource(User, '/user')` connects the `User` resource to the `/user` endpoint.

---

3. `fields` (from flask_restful)

- `fields` is used to define a structured response format for API output.
- It ensures that only the specified fields are included in the response.

Common Field Types:

- `fields.String`: Formats data as a string.
- `fields.Integer`: Formats data as an integer.
- `fields.Boolean`: Formats data as a boolean.
- `fields.List`: Formats a list of items.

Why Use `fields`?

- Prevents exposing unnecessary database fields in API responses.
- Standardizes API responses for frontend consumption.

---

4. `marshal_with` (from flask_restful)

- `marshal_with` is a decorator that automatically formats the response using predefined `fields`.
- It ensures the API response strictly follows the defined structure.

Why Use `marshal_with`?

- Avoids manually structuring JSON responses.

- Provides a clean way to enforce API response consistency.

Example Use Case:

- If an API should always return `{'id': 1, 'name': 'John', 'age': 25}`, using `marshal_with` guarantees this format.

---

5. `reqparse` (from flask_restful)

- `reqparse` is used to validate and parse request arguments (from query parameters or request bodies).
- It ensures that requests contain the required parameters and enforces data types.

Features:

- Allows specifying argument types (e.g., `int`, `str`, `float`).
- Can mark arguments as `required`, providing error messages when missing.
- Prevents the need for manual request validation.

Common Use Cases:

- Validating user inputs in a `POST` request (e.g., ensuring `username` and `password` are provided).
- Handling optional query parameters with default values.

---

6. `HTTPException` (from werkzeug.exceptions)

- `HTTPException` is a base class for handling HTTP errors in Flask.
- It provides built-in error handling for status codes like `400 Bad Request`, `404 Not Found`, `500 Internal Server Error`, etc.
- You can customize error messages and status codes by subclassing `HTTPException`.

Why Use It?

- Improves error handling by raising exceptions instead of manually returning error responses.
- Makes API responses more readable and standardized.

Example Use Cases:

- Returning a `404 Not Found` error when a resource does not exist.
- Raising a `400 Bad Request` error if input validation fails.

Module	Purpose
`Resource`	Defines API endpoints as classes with `get()`, `post()`, etc.
`Api`	Manages API routing and handles errors.
`fields`	Defines the structure of API responses.
`marshal_with`	Ensures API responses follow a specific format.
`reqparse`	Parses and validates request parameters.
`HTTPException`	Handles API errors with proper HTTP status codes.

2. Setting Up Flask-RESTful

🔹 Install Dependencies

```
pip install flask flask-restful
```

🔹 Basic Flask-RESTful API

This example creates a simple API with a single endpoint.

```
from flask import Flask
from flask_restful import Resource, Api

app = Flask(__name__)
api = Api(app)
```

```
class HelloWorld(Resource):
    def get(self):
        return {"message": "Hello, World!"}

api.add_resource(HelloWorld, "/")

if __name__ == "__main__":
    app.run(debug=True)
```

- **Run the API:**

```
python app.py
```

- **Test in browser/Postman:**

```
http://127.0.0.1:5000/
```

- **Expected Output (JSON Response)**

```
{"message": "Hello, World!"}
```

---

3. Building a REST API with CRUD Operations

- **Example: User Management API**

Step 1: Create the API

```
from flask import Flask, request
from flask_restful import Resource, Api

app = Flask(__name__)
api = Api(app)

users = {}

class User(Resource):
```

```python
    def get(self, username):
        """Retrieve user details"""
        if username in users:
            return {username: users[username]}
        return {"message": "User not found"}, 404

    def post(self, username):
        """Create a new user"""
        if username in users:
            return {"message": "User already exists"}, 400
        users[username] = request.json  # Store user data
        return {username: users[username]}, 201

    def put(self, username):
        """Update user details"""
        users[username] = request.json
        return {username: users[username]}

    def delete(self, username):
        """Delete a user"""
        if username in users:
            del users[username]
            return {"message": "User deleted"}
        return {"message": "User not found"}, 404

api.add_resource(User, "/user/<string:username>")

if __name__ == "__main__":
    app.run(debug=True)
```

Step 2: Test the API
Create User (POST)

```
curl -X POST -H "Content-Type: application/json" -d '{"age": 25, "city": "Delhi"}' http://127.0.0.1:5000/user/Alice
```

Response:

```
{"Alice": {"age": 25, "city": "Delhi"}}
```

Get User (GET)

```
curl http://127.0.0.1:5000/user/Alice
```

Response:

```
{"Alice": {"age": 25, "city": "Delhi"}}
```

Update User (PUT)

```
curl -X PUT -H "Content-Type: application/json" -d '{"age": 26,
"city": "Mumbai"}' http://127.0.0.1:5000/user/Alice
```

Response:

```
{"Alice": {"age": 26, "city": "Mumbai"}}
```

Delete User (DELETE)

```
curl -X DELETE http://127.0.0.1:5000/user/Alice
```

Response:

```
{"message": "User deleted"}
```

---

4. Adding Request Parsing & Validation

- **Use `reqparse` to validate request data:**

```
from flask_restful import reqparse

parser = reqparse.RequestParser()
parser.add_argument("age", type=int, required=True, help="Age cannot be
left blank")
parser.add_argument("city", type=str, required=True, help="City cannot be
left blank")

class User(Resource):
    def post(self, username):
        args = parser.parse_args()
```

```
        users[username] = args
        return {username: users[username]}, 201
```

2. Using `reqparse` for Handling Request Arguments

Flask-RESTful provides `reqparse` to handle incoming request data.

```
from flask import Flask
from flask_restful import Resource, Api, reqparse

app = Flask(__name__)
api = Api(app)

parser = reqparse.RequestParser()
parser.add_argument('name', type=str, required=True, help="Name cannot be
blank!")

class GreetUser(Resource):
    def get(self):
        args = parser.parse_args()
        return {"message": f"Hello, {args['name']}!"}

api.add_resource(GreetUser, '/greet')

if __name__ == '__main__':
    app.run(debug=True)
```

- Use: `GET /greet?name=John`
- If `name` is missing, it will return an error.

---

5. Returning Proper HTTP Status Codes

- **Success:**
  - `200 OK`: Successful request
  - `201 Created`: New resource created
- **Client Errors:**
  - `400 Bad Request`: Invalid input
  - `404 Not Found`: Resource does not exist

Example:

```
return {"message": "User not found"}, 404
```

---

3. Using `fields` and `marshal_with` for Formatting Output

`fields` helps structure the API response.

```
from flask import Flask
from flask_restful import Resource, Api, fields, marshal_with

app = Flask(__name__)
api = Api(app)

resource_fields = {
    'id': fields.Integer,
    'name': fields.String,
    'age': fields.Integer
}

class User(Resource):
    @marshal_with(resource_fields)
    def get(self):
        return {'id': 1, 'name': 'John Doe', 'age': 25}
```

```
api.add_resource(User, '/user')

if __name__ == '__main__':
    app.run(debug=True)
```

- Returns a structured JSON response.

---

4. Custom Error Handling Using `werkzeug.exceptions.HTTPException`

```
from flask import Flask
from flask_restful import Resource, Api
from werkzeug.exceptions import HTTPException

app = Flask(__name__)
api = Api(app)

class CustomError(HTTPException):
    code = 400
    description = "This is a custom error message."

class ErrorExample(Resource):
    def get(self):
        raise CustomError

api.add_resource(ErrorExample, '/error')

if __name__ == '__main__':
    app.run(debug=True)
```

- Accessing `/error` will trigger a **400 Bad Request** error with a custom message.

---

6. Handling Authentication with Flask-JWT

Install Flask-JWT:

```
pip install flask-jwt
```

🔹 **Example: Protect an endpoint using JWT**

```
from flask_jwt import JWT, jwt_required
from werkzeug.security import safe_str_cmp

users = [{"id": 1, "username": "admin", "password": "password"}]

def authenticate(username, password):
    user = next((u for u in users if u["username"] == username), None)
    if user and safe_str_cmp(user["password"], password):
        return user

def identity(payload):
    user_id = payload["identity"]
    return next((u for u in users if u["id"] == user_id), None)

app.config["SECRET_KEY"] = "supersecretkey"
jwt = JWT(app, authenticate, identity)

class Protected(Resource):
    @jwt_required()
    def get(self):
        return {"message": "You have accessed a protected route"}

api.add_resource(Protected, "/protected")
```

[redacted]

Testing Authentication

Login to get token

```
curl -X POST -H "Content-Type: application/json" -d '{"username": "admin", "password": "password"}' http://127.0.0.1:5000/auth
```

Response:

```
{"access_token": "your_token_here"}
```

Access Protected Route

```
curl -H "Authorization: JWT your_token_here" http://127.0.0.1:5000/protected
```

1. **Response:**
   ```
   {"message": "You have accessed a protected route"}
   ```

https://flask-restful.readthedocs.io/en/latest/